Assignment 1

Name: Wamsambiang Laloo
Roll No.: B22CS015
Signature: Laloo
Branch: CSE
Course Code: CS 418

1. Explain Good Turing Smoothing and Kneser Ney Smoothing with Suitable examples (4+4=8)

Ans Good Turing Smoothing:

Good Turing Smoothing is a technique used in language modeling to handle rare and unseen words. In MLE, the probability of a word is calculated using its frequency. If a word does not appear in the training data, its probability becomes zero. This is a problem because if one word ~~(zero)~~ has zero probability, the entire sentence probability becomes zero. Good Turing Smoothing solves this problem by adjusting the word counts. Instead of using the original count c, it estimates a new adjusted count based on how many words occur once, twice, etc. It also reserves some probability mass for unseen words.

For Example, Suppose a corpus contains 100 total words and 10 words appear only once. According to Good-Turing

the probability assigned to unseen word is :

$$P(\text{unseen}) = \frac{N_1}{N} = \frac{10}{100} = 0.1$$

This means 10% probability is reserved for unseen words. Thus, it reduces the probability of frequent words slightly and assigns some probability to unseen words, making the model more realistic.

Kneser Ney Smoothing:

Kneser Ney Smoothing is an advanced smoothing technique used in n-gram models. It improves over simple discounting methods by considering not only how often a word appears, but also in how many different contexts it appears.

For Example: the word "Francisco" may appear many times in a dataset but mostly after the word "San" (as in "San Francisco"). If we consider frequency only, then Francisco would get high probability everywhere. However, it rarely appears in other contexts like "New Francisco" Kneser-Ney solves this by using continuation probability, which depends on the number of different contexts a word appears in. It also subtracts a small discount from counts and redistributes that probability to lower-order models. Thus, Kneser Ney Smoothing uses discounting, considers context diversity, provides better probability estimates and it is considered one of the

best Smoothing techniques for n-gram models.

2. Prepare Minimum Edit Distance (Levenshtein) table between the words INTENTION to EXECUTION. (7)

Ans To compute the Minimum Edit Distance (Levenshtein) between the words INTENTION and EXECUTION, we use the dynamic programming method. In Levenshtein distance, the allowed operations are:

- Insertion (cost = 1)
- Deletion (cost = 1)
- Substitution (cost = 1 if characters are different, 0 if same)

First, we construct a matrix of size $(n+1) \times (m+1)$,
where n = length of intention = 9
m = length of execution = 9
So the table size will be 10 x 10

Step 1: Initialization
If one string is empty:
- $D(i,0) = i$ (delete all characters)
- $D(0,j) = j$ (insert all characters)

Thus, the first row and first column are filled from 0-9

Step 2: Recurrence formula
For each cell:

$$D(i,j) = \min \begin{cases} D(i-1,j)+1 & \text{(deletion)} \\ D(i,j-1)+1 & \text{(insertion)} \\ D(i-1,j-1)+\text{cost} & \text{(substitution)} \end{cases}$$

where cost = 0 if characters match, otherwise 1.

Minimum Edit Distance Table:

	ϕ	I	N	T	E	N	T	I	O	N
ϕ	0	1	2	3	4	5	6	7	8	9
E	1	1	2	3	3	4	5	6	7	8
X	2	2	2	3	4	4	5	6	7	8
E	3	3	3	3	3	4	5	6	7	8
C	4	4	4	4	4	4	5	6	7	8
U	5	5	5	5	5	5	5	6	7	8
T	6	6	6	5	6	6	5	6	7	8
I	7	6	7	6	6	7	6	5	6	7
O	8	7	7	7	7	7	7	6	5	6
N	9	8	7	8	8	7	8	7	6	5

Therefore, the minimum edit distance between "INTENTION" and "EXECUTION" is 5.